

内容

1. 在 ppForm 和 BSpline 格式下实现 S_1^0 和 S_3^2 。
2. 在 ppForm 条件下推导（概述）并实现了三次样条函数 S_3^2 : 周期边界条件，自然边界条件，完全边界条件（任意节点）。
3. 在 BSpline 条件下推导（概述）并实现了三次样条函数 S_3^2 : 周期边界条件，自然边界条件，完全边界条件（任意节点）。
4. 验证在 ppForm 和 BSpline 条件下，三次样条函数 S_3^2 是相同的（在 problemA 中验证了）。
5. 验证了 BSpline 格式下支持任意阶任意节点的样条绘制。（任意阶只要给出合适的边界条件都可以获得良好的拟合曲线，任意节点在 problemE 中使用 cumulative chordal length 时已经验证）
6. 实现平面上的样条曲线拟合。（problemE 中的心形线与螺线）
7. 实现球面上的样条曲线拟合。（problemE 中的三维样条曲线）
8. 实现了书上的问题。

附加分数

1. 任意阶的样条曲线绘制。
2. 误差分析。
3. 更多例子验证。

ppForm 实现 S_1^0

概述

因为 S_1^0 的定义为分段线性函数，所以只需要对每个分段进行线形插值即可。

实现

1. 首先，在构造函数 ppForm 中加入一个 bool 型的参数表明此时构造的是线性样条。

```
1 ppForm(const Function& f, double a, double b, int nodeCount, bool isLinear)
2 : f(f), nodeCount(nodeCount);
```

2. 接下来，实现线性插值并返回插值结果。

```
1 double LinearInterpolation(double x);
```

ppForm 实现 $\mathbb{S}_3^2$

概述

想要实现 $\mathbb{S}_3^2$ ，需要对每个分段线性函数进行三次样条插值。

首先，我们记 $S_i(x)$ 为函数在 $x \in [x_i, x_{i+1}]$ 处的插值函数，记 $h_i = x_{i+1} - x_i$, $S''(x_i) = M_i$, $i = 0, 1, \dots, n$ 。由于 $S''(x)$ 的线性性，我们有

$$S''_i(x) = \frac{x_{i+1} - x}{h_i} M_i + \frac{x - x_i}{h_i} M_{i+1}$$

对上述等式两边做两次积分，我们有

$$S_i(x) = \frac{(x_{i+1} - x)^3}{6h_i} M_i + \frac{(x - x_i)^3}{6h_i} M_{i+1} + A_i(x - x_{i+1}) + B_i$$

利用插值条件 $S_i(x_i) = f_i$, $S_i(x_{i+1}) = f_{i+1}$ ，我们有

$$A_i = \frac{f_{i+1} - f_i}{h_i} - \frac{h_i}{6}(M_{i+1} - M_i), \quad B_i = f_{i+1} - \frac{h_i^2}{6}M_{i+1}$$

从而有，

$$S_i(x) = \frac{(x_{i+1} - x)^3}{6h_i} M_i + \frac{(x - x_i)^3}{6h_i} M_{i+1} + \left(f_i - \frac{M_i}{6}h_i^2\right) \frac{x_{i+1} - x}{h_i} + \left(f_{i+1} - \frac{M_{i+1}}{6}h_i^2\right) \frac{x - x_i}{h_i}$$

接下来我们求解 M_i 即可确定三次样条函数的表达式，需要用到样条一阶导连续的性质。首先对 $S_i(x)$ 求导，

$$S'_i(x) = -\frac{(x_{i+1} - x)^2}{2h_i} M_i + \frac{(x - x_i)^2}{2h_i} M_{i+1} + \frac{f_{i+1} - f_i}{h_i} + \frac{M_{i+1} - M_i}{6}h_i$$

利用一阶导连续性，有

$$\frac{h_i}{6}M_i + \frac{h_i + h_{i+1}}{3}M_{i+1} + \frac{h_{i+1}}{6}M_{i+2} = f[x_{i+1}, x_{i+2}] - f[x_i, x_{i+1}]$$

上是两端同时除以 $(h_{i+1} + h_i)/6$ ，有

$$\mu_{i+1}M_i + 2M_{i+1} + \lambda_{i+1}M_{i+2} = b_{i+1}$$

其中，

$$\mu_{i+1} = \frac{h_i}{h_i + h_{i+1}}, \lambda_{i+1} = \frac{h_{i+1}}{h_i + h_{i+1}}, d_{i+1} = 6f[x_i, x_{i+1}, x_{i+2}]$$

结合两个边界条件即可求出 M_i 。

实现

- 首先，定义一个枚举类 SplineType，用于表示使用三次样条的边界条件。

```
1  enum class SplineType{
2      Natural,
3      Complete,
4      Periodic
5  };
```

- 在构造函数中初始化插值点以及插值点的函数值。平均插值点和任意插值点调用不同的构造函数。

平均插值点和任意点的构造函数如下：

```

1  ppForm(const Function& f, double a, double b, int nodeCount, SplineType
    splineType = SplineType:: Natural);
2  ppForm(const Function& f, std::vector<double> nodes, SplineType splineType
    = SplineType:: Natural)

```

3. 进入 useFunction 函数, 根据传入的枚举类型确定三次样条的边界条件。

```

1  void useFunction()

```

4. 函数 calMiu(int i) 用于计算 μ_i , 函数 calNabuda(int i) 用于计算 λ_i 。

```

1  double calMiu(int i);
2  double calNabuda(int i);

```

5. 计算二阶差商 $f[x_i, x_{i+1}]$ 和三阶差商 $f[x_{i-1}, x_i, x_{i+1}]$ 。

```

1  double calQuadraticDivided(int i);
2  double calCubicDivided(int i);

```

6. 求解三对角矩阵的解。

```

1  void caltridiagonal();

```

7. 根据给定边界条件, 调用相应的函数, 完成矩阵方程组的初始化, 并调用 caltridiagonal 函数求解三对角矩阵的解, 即 M_i 。

```

1  void computeNaturalSpline();
2  void computeCompleteSpline();
3  void computePeriodicSpline();
4  void caltridiagonal();

```

8. 最后, 我们计算插值点的函数值。

```

1  double getValue(double x);

```

BSpline

概述

根据书上内容, 只需写出 BSpline 基函数, 求出基函数的系数, 即可求出插值函数。

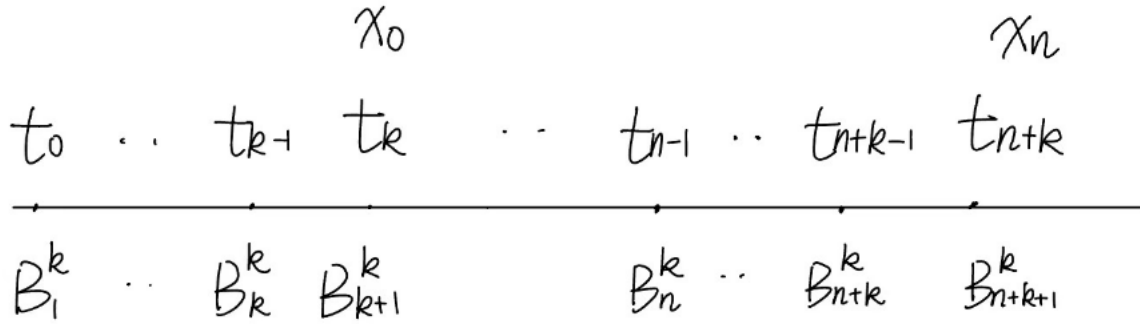
$$B_i^k(x) = \frac{x - t_{i-1}}{t_{i+k} - t_{i-1}} B_{i-1}^{k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k} - t_i} B_{i+1}^{k-1}(x), \quad (1)$$

$$B_i^k \geq 0, \quad x \in [t_{i-1}, t_{i+k}]. \quad (2)$$

其中, t_i 为节点, k 为次数。

由于 BSpline 基函数的支撑性, 我们需要扩展插值点 $\{x_i\}_{i=0}^n$ 为节点, 使得基函数的支撑集包含插值点, 记节点集合为 $T = \{t_0, t_1, \dots, t_m\}$ 。其中, $m = n + k, x_i = t_{i+k}$ 。

考虑插值基函数的支撑性, 在插值边界 $x_0 = t_k, x_n = t_{n+k}$ 处,



如图所示, $B_{k+1}^k(x_0) = 0, B_{n+k+1}^k(x_n) = 0$, 所以对每个节点 x_i , 对其作用的基函数有 $B_{i-k+1}^k(x), B_{i-k+2}^k(x), \dots, B_i^k(x)$ 。对于待拟合函数 $f(x)$, 我们可以将其表示为

$$f(x) = \sum_{i=1}^m P_i B_i^k(x), \quad (3)$$

$$f(x_j) = \sum_{i=1}^m P_i B_i^k(t_{j+k}) = \sum_{i=i-k+1}^i P_i B_i^k(t_{j+k}), \quad (4)$$

$$(5)$$

写成矩阵的形式即

$$\begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix} \begin{pmatrix} B_1^k(t_k) & B_2^k(t_k) & \cdots & B_k^k(t_k) & 0 & 0 \\ 0 & B_2^k(t_{k+1}) & \cdots & B_{k+1}^k(t_{k+1}) & 0 & 0 \\ \vdots & \vdots & & \vdots & & \\ 0 & \cdots & B_{n+1}^k(t_{n+k}) & B_{n+2}^k(t_k) & \cdots & B_{n+k}^k(t_{n+k}) \end{pmatrix} = \begin{pmatrix} P_1 \\ P_2 \\ \vdots \\ P_{n+k} \end{pmatrix}$$

显然系数矩阵是一个 $(n+1) \times (n+k)$ 阶的矩阵, 其中 P_i 为待拟合函数的系数, 要求解还需要提供 $k-1$ 个边界条件。

$\mathbb{B}_3$

考虑三次 B 样条的边界条件: 自然, 完全, 周期。

1. 自然边界条件: $f''(x_0) = f''(x_n) = 0$ 。利用课本中的 B 样条导数公式和 (3), 当 $k=3$ 时, 有

$$f''(x) = \sum_{j=1}^m \left[\frac{6}{t_{j+2} - t_{j-1}} \left(\frac{B_j^1(x)}{t_{j+1} - t_{j-1}} - \frac{B_{j+1}^1(x)}{t_{j+2} - t_j} \right) - \frac{6}{t_{j+3} - t_j} \left(\frac{B_{j+1}^1(x)}{t_{j+2} - t_j} - \frac{B_{j+2}^1(x)}{t_{j+3} - t_{j+1}} \right) P_j \right] \quad (6)$$

$$(7)$$

$$B_i^1(x) = \begin{cases} \frac{x-t_{i-1}}{t_i-t_{i-1}}, & x \in (t_{i-1}, t_i] \\ \frac{t_{i+1}-x}{t_{i+1}-t_i}, & x \in (t_i, t_{i+1}) \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

分别带入 x_0, x_n , 即可得到边界条件。

2. 完全边界条件: $f'(x_0) = f'_{x_0}, f'(x_n) = f'_{x_n}$ 。利用课本中的 B 样条导数公式和 (3), 当 $k = 3$ 时, 有

$$f'(x) = \sum_{j=1}^m 3 \left[\frac{B_j^2(x)}{t_{j+k-1} - t_{j-1}} - \frac{B_{j+1}^2(x)}{t_{j+k} - t_j} \right] P_j \quad (9)$$

(10)

带入 x_0, x_n , 即可得到边界条件。

3. 周期边界条件: $f(x_0) = f(x_n), f'(x_0) = f'(x_n), f''(x_0) = f''(x_n)$ 。只需使用上面的公式即可获得。

实现

BSpline 类

1. 成员变量如下所示:

```

1      private:
2      int k; //degree of bspline
3      const Function& f;
4      int nodeCount;
5      int n; //n是插值点个数减一
6      std::vector<double> nodes; //插值点
7      std::vector<double> values;
8      std::vector<double> P; //插值系数
9      std::vector<double> t; //节点
10     std::vector<std::vector<double>>> A; //系数矩阵
11     SplineType splineType; //插值类型

```

2. BSpline 有两个构造函数, 分别用于实现均匀节点和任意给定节点的 BSpline。

```

1      BSpline(int k, const Function& f, double a, double b, int nodeCount,
              SplineType splineType);
2      BSpline(int k, const Function& f, std::vector<double> nodes, SplineType
              splineType);

```

3. 初始化系数矩阵, 添加边界条件, 以及求解插值系数如下所示:

```

1      double constructBSpline(int i, int k, double x);
2      void constructMatrix();
3      void addrange();
4      void addNaturalCondition();
5      void addCompleteCondition();
6      void addPeriodicCondition();
7      void solve();

```

4. 计算插值函数的值如下所示:

```

1      double getValue(double x);

```

Problem

A

linear ppForm Spline

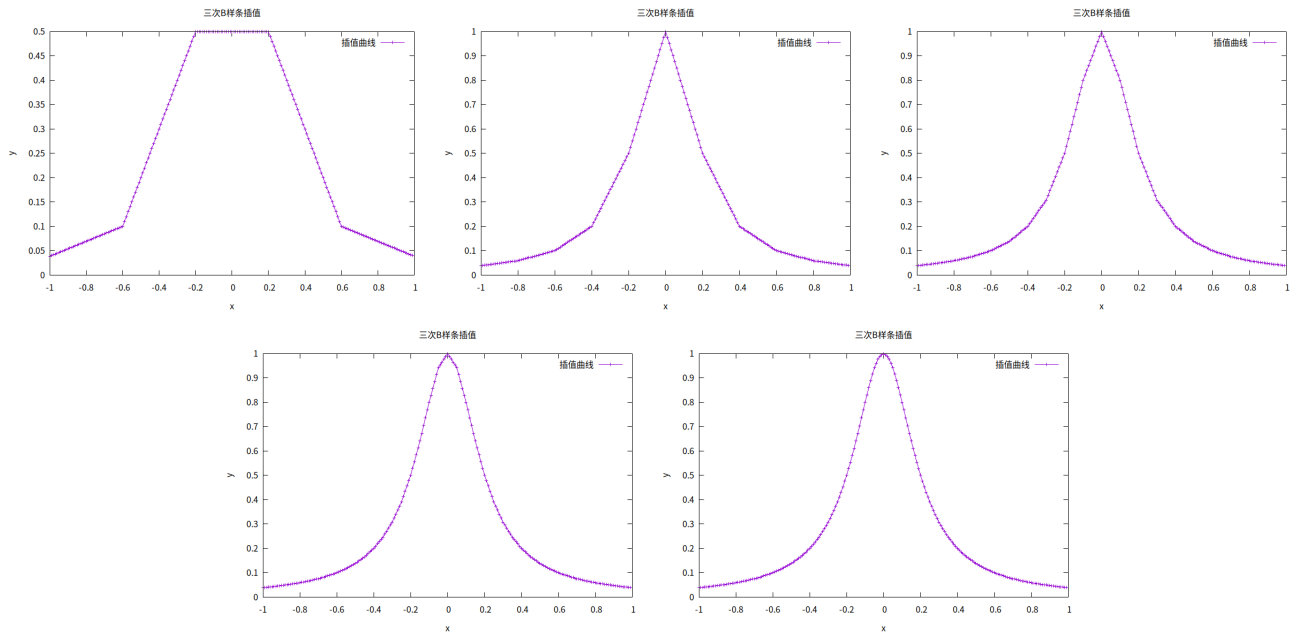


图 1: $N=6$, $N=11$, $N=21$, $N=41$, $N=81$

linear BSpline

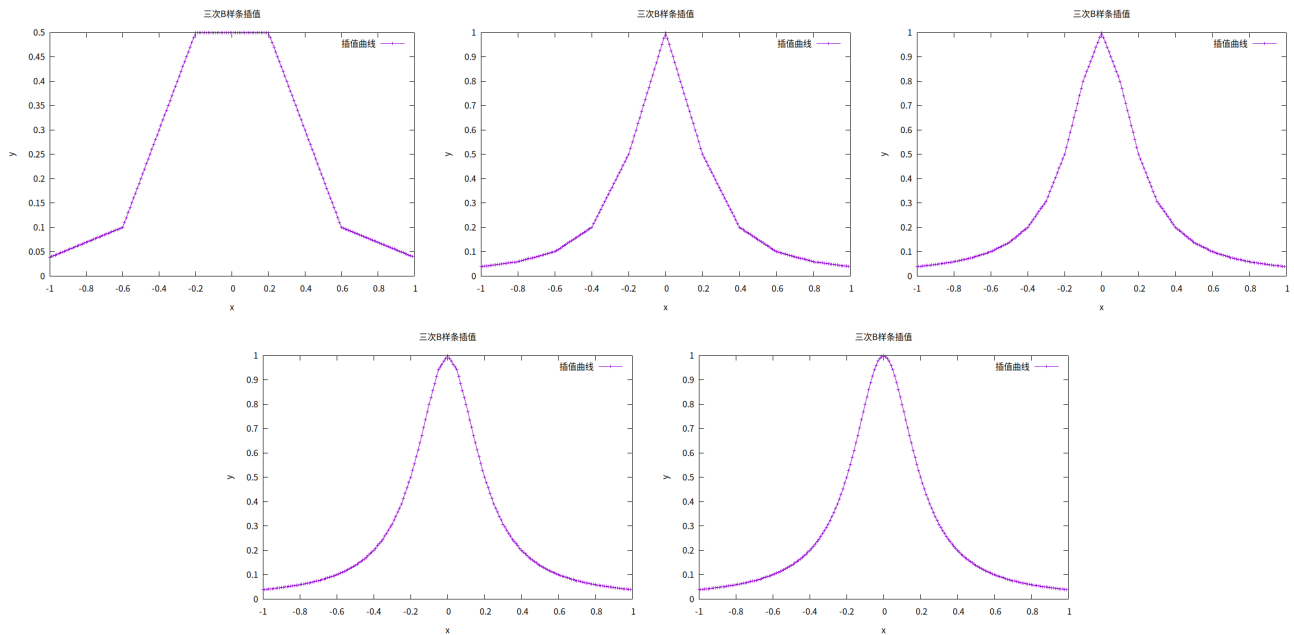


图 2: $N=6$, $N=11$, $N=21$, $N=41$, $N=81$

Natural ppForm Spline

```
1 cubic-ppForm_Spline_Error
2 N = 6 -> Errormax: 0.423856
```

```

3 N = 11 -> Errormax: 0.042903
4 N = 21 -> Errormax: 0.0136586
5 N = 41 -> Errormax: 0.00371266
6 N = 81 -> Errormax: 0.000961739

```

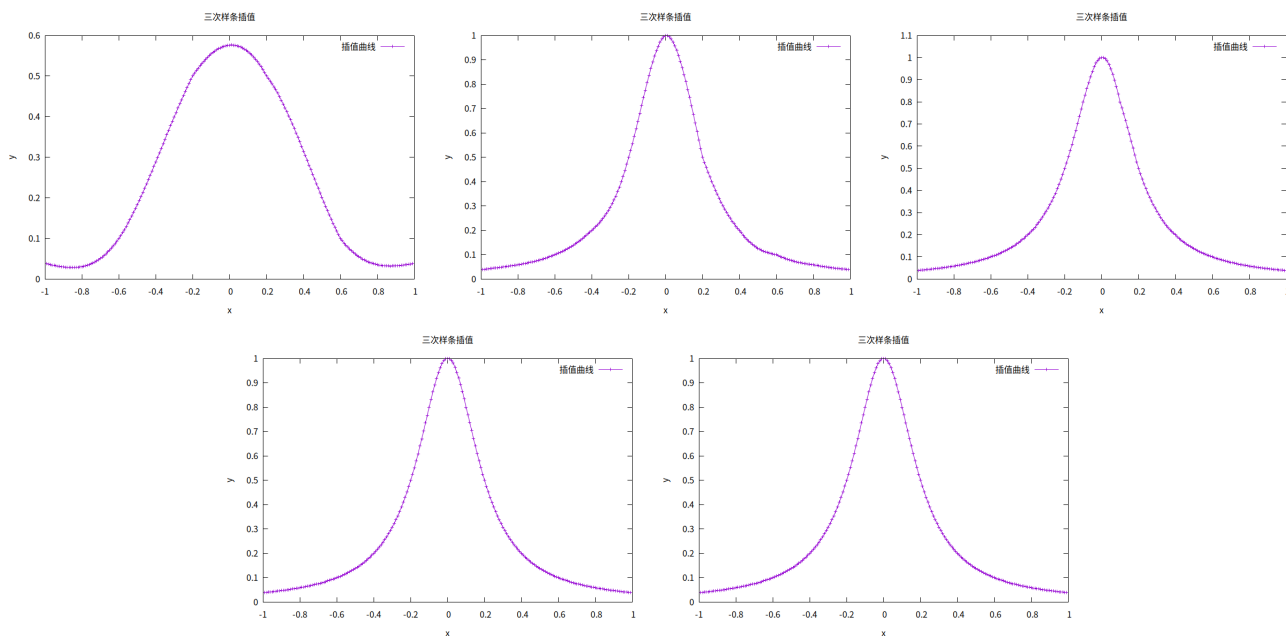


图 3: N=6, N=11, N=21, N=41, N=81

Complete ppForm Spline

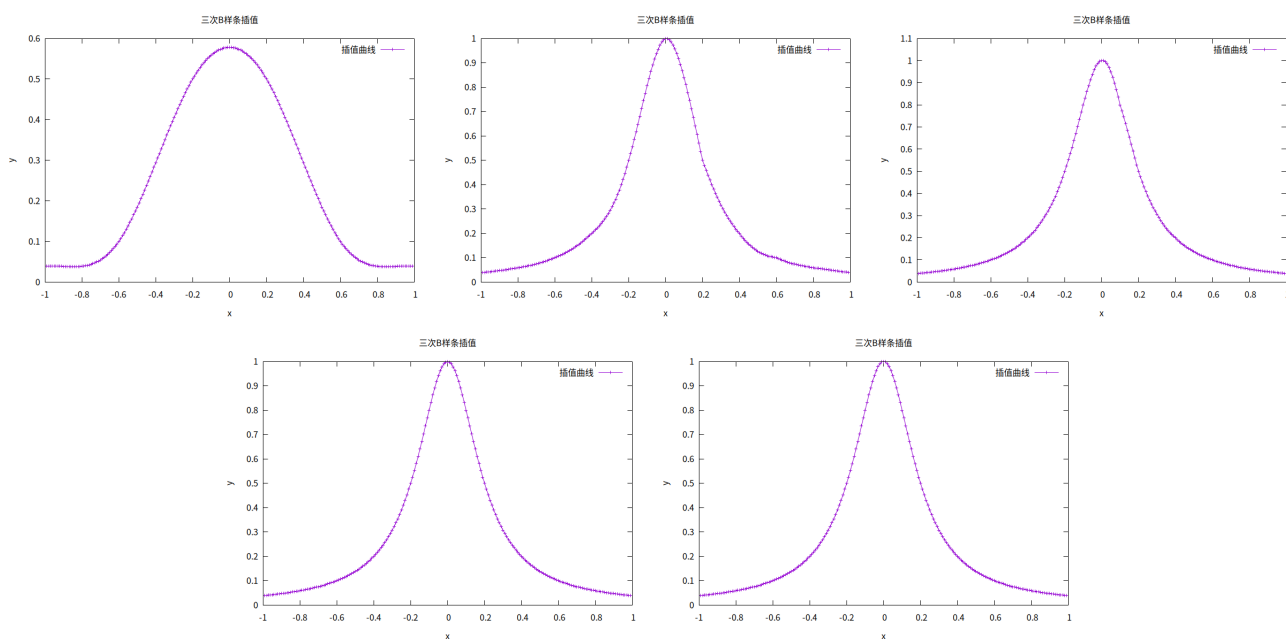
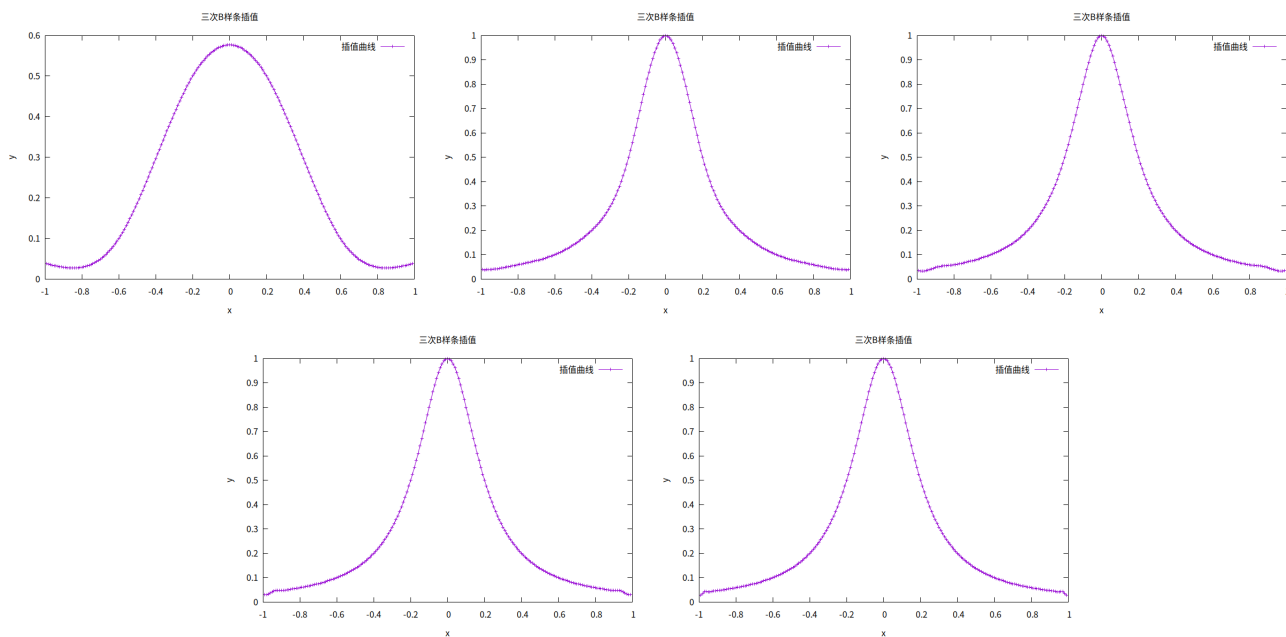
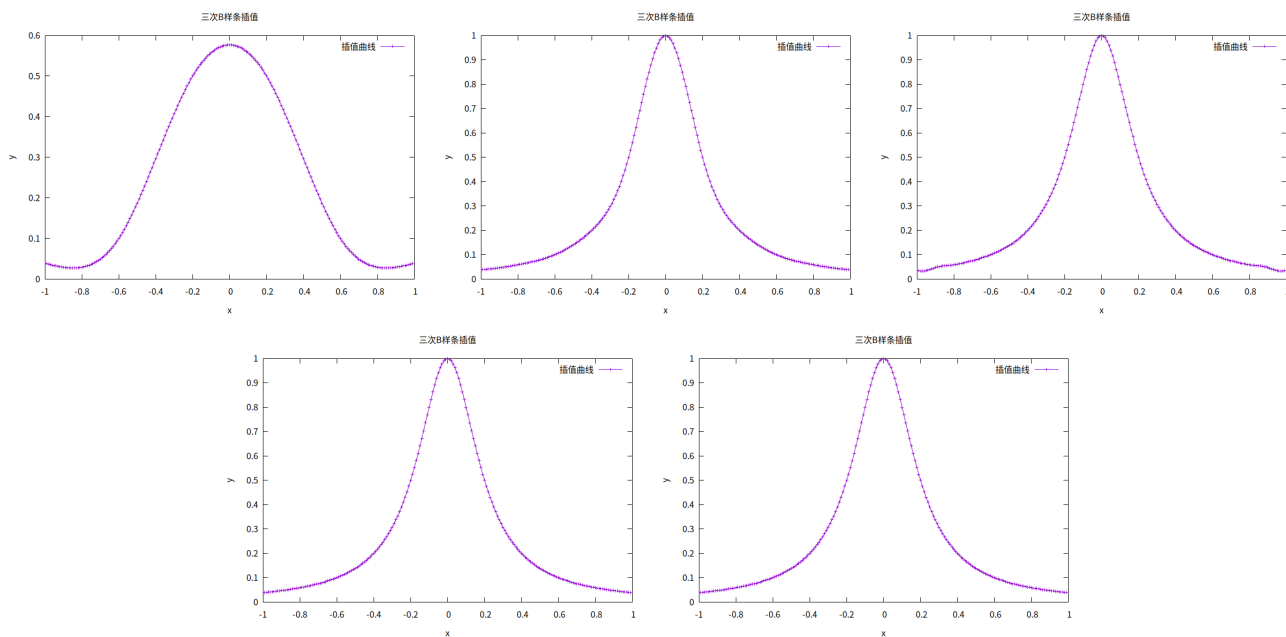


图 4: N=6, N=11, N=21, N=41, N=81

Natural BSpline

图 5: $N=6, N=11, N=21, N=41, N=81$

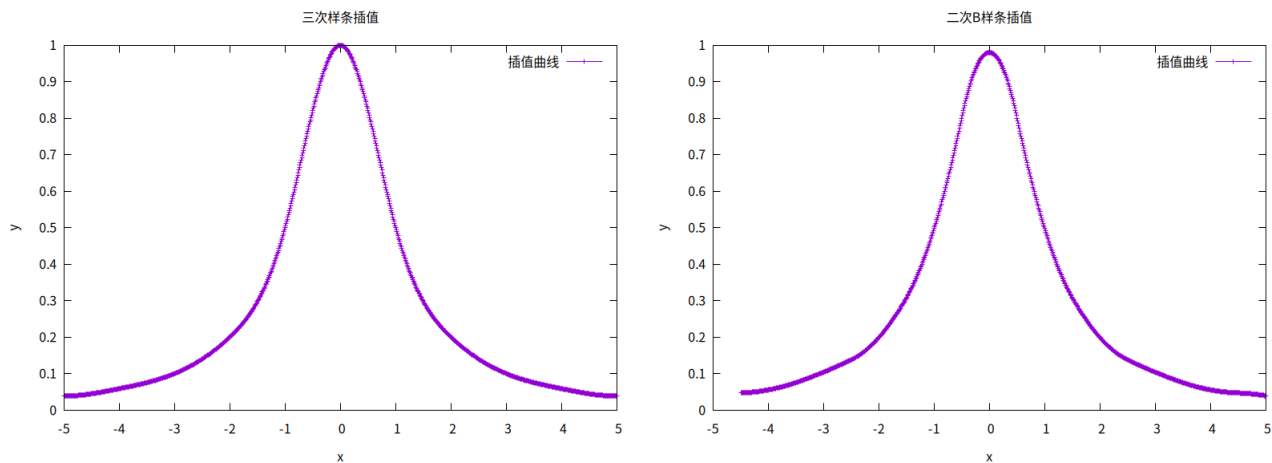
Complete BSpline

图 6: $N=6, N=11, N=21, N=41, N=81$

验证发现 ppform 和 BSpline 在相同边界条件下获得的插值曲线基本相同。

C

拟合结果如图所示：

图 7: S_3^2, S_2^1

分别按题意要求画出拟合曲线（左右）。

D

S_3^2

```

1 x = -3.5 -> Error: 0.000460179
2 x = -3 -> Error: 5.55112e-17
3 x = -0.5 -> Error: 0.0205259
4 x = 0 -> Error: 0
5 x = 0.5 -> Error: 0.0205259
6 x = 3 -> Error: 5.55112e-17
7 x = 3.5 -> Error: 0.000460179

```

S_2^1

```

1 x = -3.5 -> Error: 4.16334e-17
2 x = -3 -> Error: 0.00397321
3 x = -0.5 -> Error: 0
4 x = 0 -> Error: Error: 0.0203838
5 x = 0.5 -> Error: 0
6 x = 3 -> Error: 0.00397321
7 x = 3.5 -> Error: 4.16334e-17

```

有些点的误差接近机器精度的原因是那些点就是用于拟合的插值点，因此误差为 0 或极小。

E

实现

为了计算 cumulative chordal length，我们需要计算每个节点的长度，并将其累加。因为一些函数的 $r(s)$ 表达式复杂或无法求得，我的做法是先手算出 $s(t) = \int_0^t \|r'(t)\|_2 dx$ 的表达式，将这个表达式写成一个 Function 类 fs (Function 类中定义了二分法)，在 fs 中重载运算符 +，用于应用二分法求出每小段弧长对应的 t 值。

```

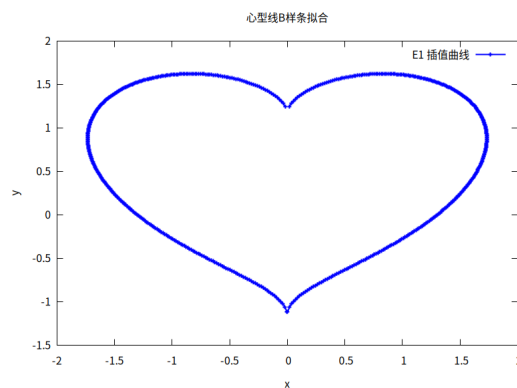
1 class Function{
2 public:
3     virtual double operator()(double x) const = 0;
4     virtual double derivative(double x) const;
5     virtual ~Function(){}
6     double bisection_method(double a, double b, double tolerance = 1e-6, int
        max_iterations = 100);
7 };
8
9 class f2s:public Function{
10 public:
11     double operator()(double x);
12     Function* operator+(double value) {
13         class AddedFunction : public Function{
14         public:
15             const f2s& baseFunction;
16             double additionalValue;
17             AddedFunction(const f2s& f, double v): baseFunction(f), additionalValue(
                v);
18             double operator()(double x) const override;
19             return new AddedFunction(*this, value);
20         }
21     }
22 }

```

Function 类的具体代码在 Function.h 中, f2s 在 problem.cpp 中。

心形线

原图



本题与前面的样条拟合有所不同。该题要求完成绘制的是一个闭合曲线，但是注意到心形线关于 y 轴对称，故

只需拟合图片的左半部分，完成后对称得到右半部分。将函数写成参数形式：

$$\begin{cases} x(t) = \sqrt{3} \sin(t) \\ y(t) = \frac{2}{3} \left(\sqrt{3} \cos(t) + \sqrt{\sqrt{3} \sin(t)} \right) \end{cases}$$

以下是使用等距节点拟合心形线的结果：

Natural

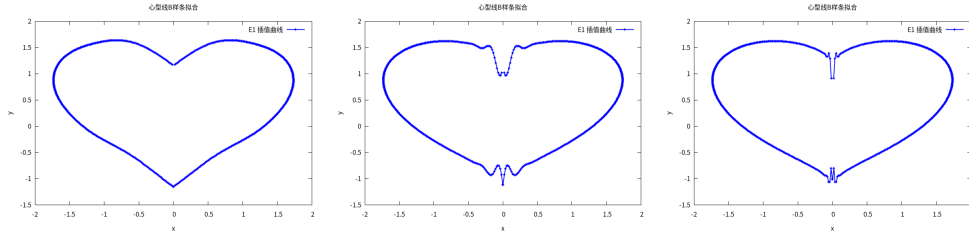


图 8: N=10, N=40, N=160

Complete

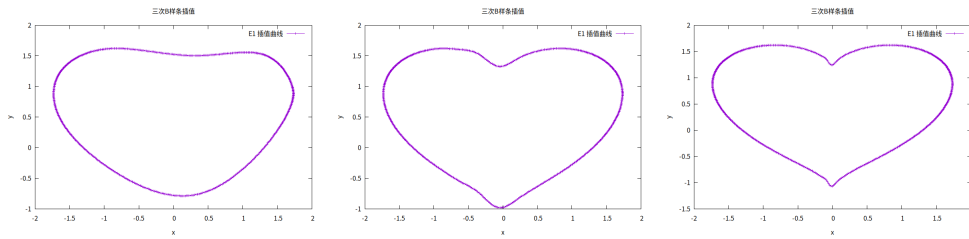


图 9: N=10, N=40, N=160

Periodic

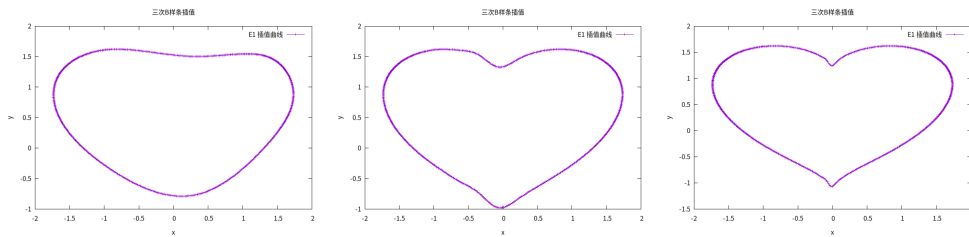
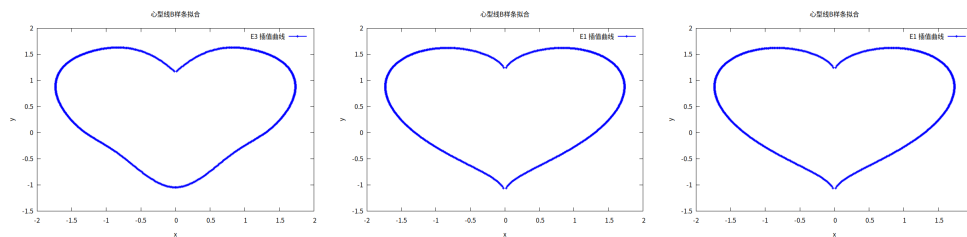


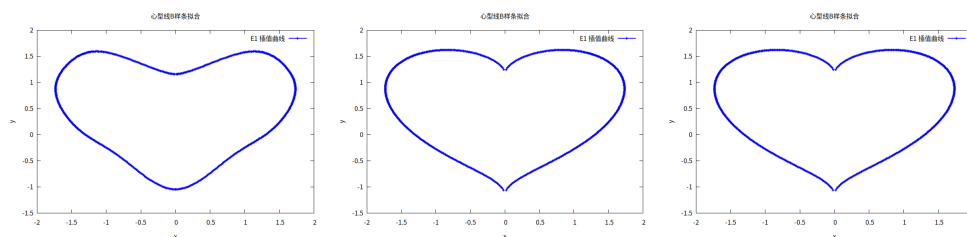
图 10: N=10, N=40, N=160

发现 Natural 条件下，由于在边界点 $0, \pi$ 处函数不可导，导致在边界处插值函数值不准确。解决办法是应用 cumulative chordal length 方法在边界处使用更多节点，加强 b 样条在边界处的准确性。

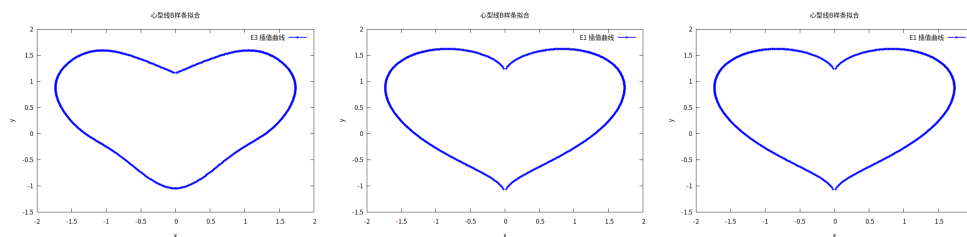
结果如下：**Natural**

图 11: $N=10$, $N=40$, $N=160$

Complete

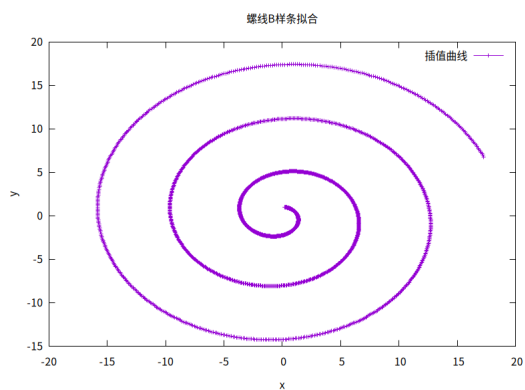
图 12: $N=10$, $N=40$, $N=160$

Periodic

图 13: $N=10$, $N=40$, $N=160$

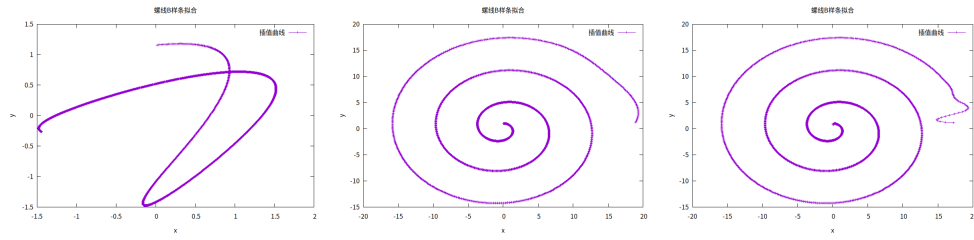
螺线

原图

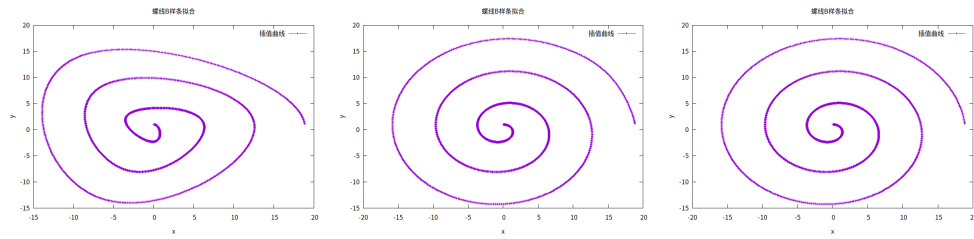


本题给出的函数是光滑的，因此直接拟合即可。

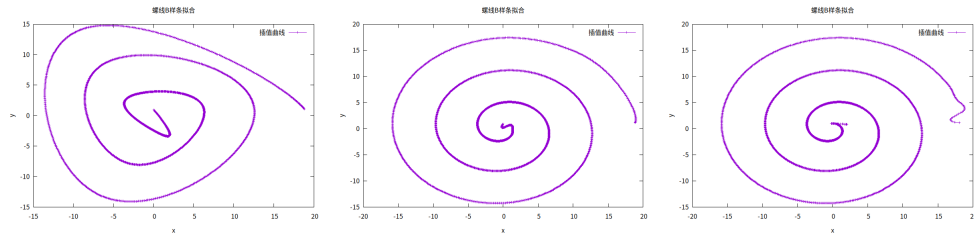
等距节点的拟合结果如下：**Natural**

图 14: $N=10$, $N=40$, $N=160$

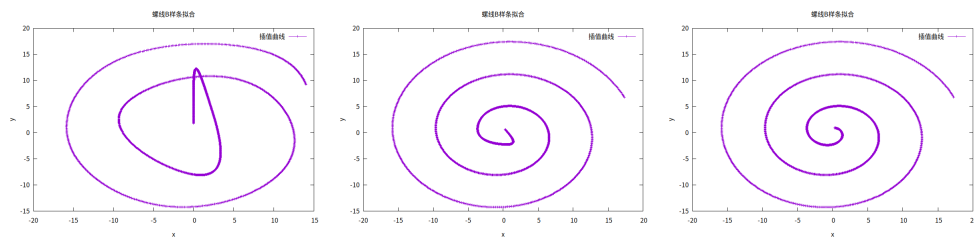
Complete

图 15: $N=10$, $N=40$, $N=160$

Periodic

图 16: $N=10$, $N=40$, $N=160$

Cumulative chordal length 做法先求出函数在 $[0, 6\pi]$ 的总长度，平均分成 N 段长度为 h ，对每个 h 利用二分法求出对应的 t 值，再用 t 值计算插值函数的值。结果如下：**Natural**

图 17: $N=10$, $N=40$, $N=160$

Complete

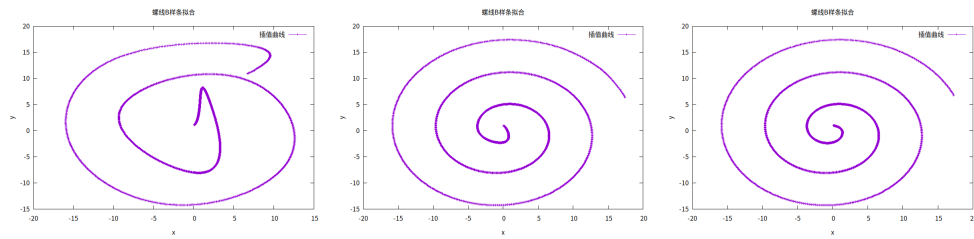


图 18: N=10, N=40, N=160

Periodic

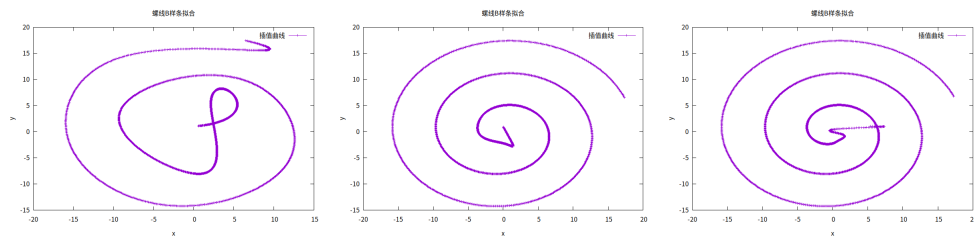
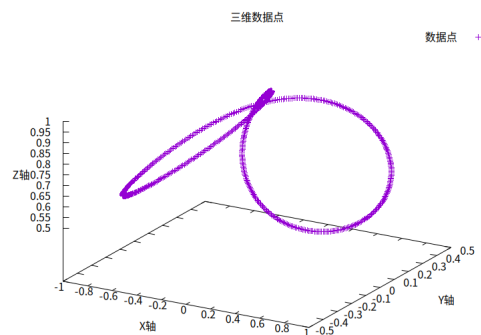


图 19: N=10, N=40, N=160

三维曲线

原图



因为求出弦长后发现 $s(t) = t$, 因此等距节点的结果就等于 Cumulative chordal length 的结果。

Natural

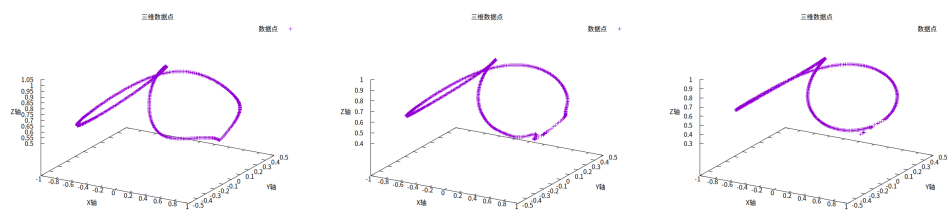


图 20: N=10, N=40, N=160

Complete

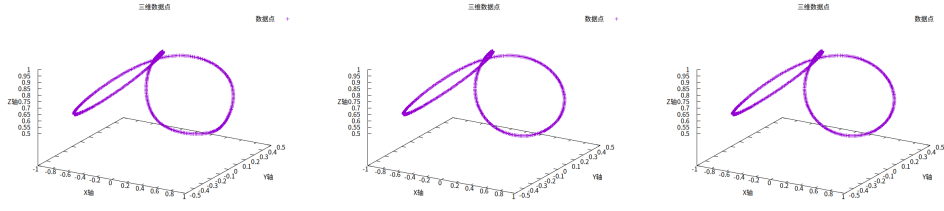


图 21: N=10, N=40, N=160

Periodic

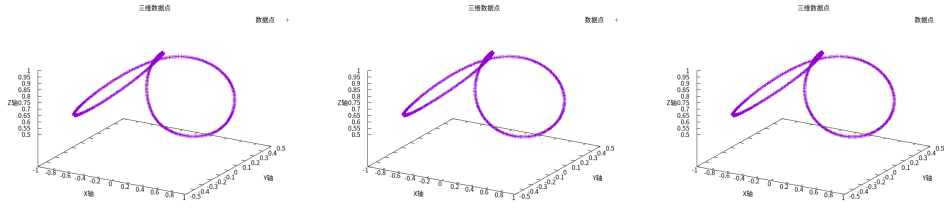
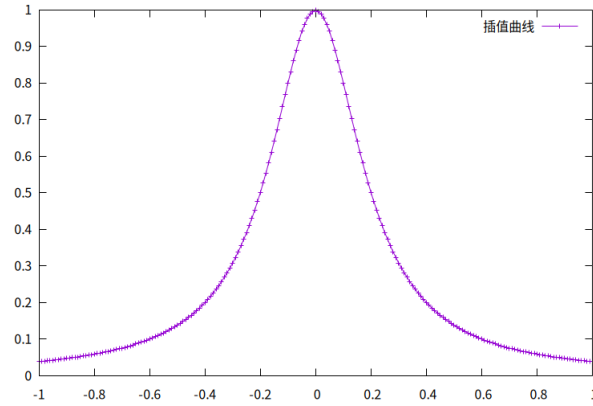


图 22: N=10, N=40, N=160

任意阶的样条

因为当阶数增加之后边界条件也需要增加，我测试了七次样条，对函数 $\frac{1}{1+x^2}$ 进行拟合结果如下：



误差分析

Complete ppForm Spline

三次完全样条 $S(x)$ 是一个分段三次多项式，在每个区间上 $[x_i, x_{i+1}]$ 可以表示为

$$S(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

由误差公式可得

$$f(x) - S(x) = \frac{-1}{(2k)!} f^{(2k)}(\xi) \prod_{i=0}^k (x - x_i)$$

其中 k 是阶数， $\xi \in [x_0, x_n]$.

考虑到三次样条，在每个分段 $[x_i, x_{i+1}]$ 上，误差表达式为 $f(x) - S(x) = \frac{-1}{4!} f^{(4)}(\xi)(x - x_i)^4$ ，考虑无穷范数，有 $\|f(x) - S(x)\|_{\infty} \leq \frac{h^4}{4!} \|f^{(4)}(x)\|_{\infty}$,

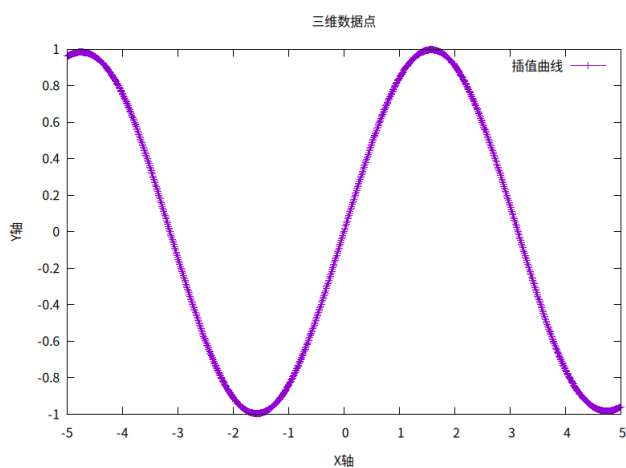
假设 $f(x)$ 在 $[x_0, x_n]$ 上四阶可导, 在每个分段上误差满足 $|f(x) - S(x)| = O(h^4)$ 其中 $h = \max_i |x_{i+1} - x_i|$ 当考虑全局误差时, 由于样条的分段特性和光滑性, 误差的主项累计在全局上最多仍为 $O(h^4)$, 因此全局误差为:

$$\|f(x) - S(x)\|_{\infty} = O(h^4)$$

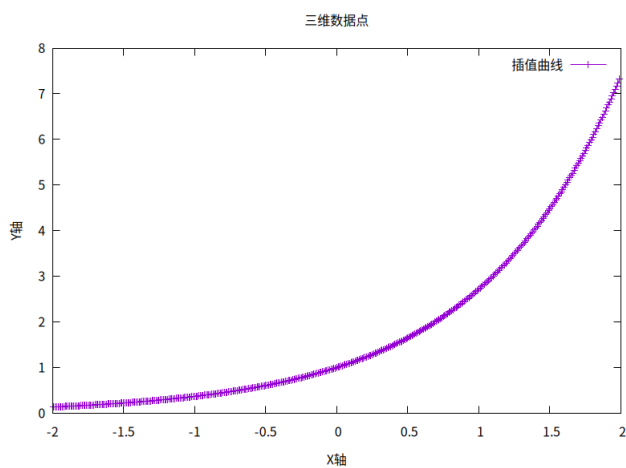
如果函数不够光滑, 实际收敛阶会低于 4. 如果使用非均匀节点会使局部误差增大, 影响全局误差分布。

更多例子

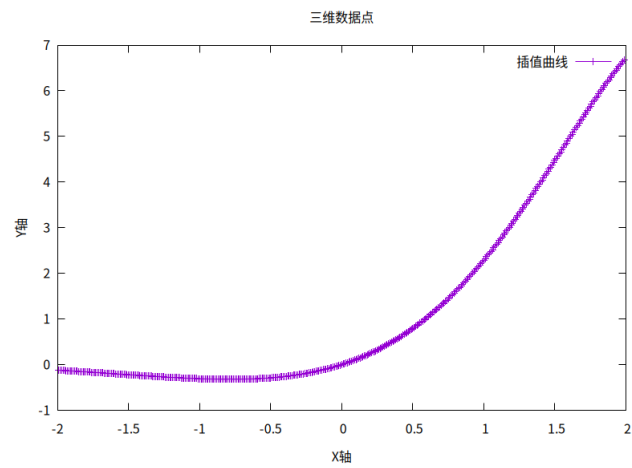
Sinx



e^x



$e^x \sin(x)$



发现这些更多例子都能够良好拟合。